# 煤矿应急救援智能体实施方案（基于 DSH）

Oct 27, 2026 · @Jeff

## 项目概述

本项目以 DeepSeek Harness（DSH）为底座，开发煤矿应急救援智能体，为火灾、水灾、坍塌等事故提供灾情研判、物资调度和处置方案三项辅助决策能力。

建设路线是**先做通一个闭环，再横向扩展**。第一期只做采空区煤自燃，把预警、防灭火处置、应急响应、火区管理和复盘全流程跑通。验证可行后，复用同一套数据底座、MCP 服务和智能体框架，依次扩展到带式输送机巷道火灾、水灾和坍塌。

**推进方式**：在没有真实矿井可用的情况下，先建设一座模拟矿井，用模拟数据把闭环跑通并向客户演示；客户认可后再接入真实矿井，按模块把模拟数据源逐个替换为真实系统，没有现成系统的模块新开发。开发、演示、部署共用同一套智能体。

- **定位**：辅助决策。所有调度和处置指令由指挥员确认后执行，系统不直接下达指令。
- **部署**：矿区内网私有化部署，模型由其他部门在本地部署，数据不出网；开发阶段使用 DeepSeek API，只处理模拟数据和公开资料。
- **试点范围**：先建 1 座模拟矿井，客户认可后接入 1 座真实矿井；1 类场景（采空区煤自燃）。
- **一期周期**：约 6 个月完成模拟矿井闭环和客户演示；接入真实矿井的周期视客户条件而定。

## 为什么选 DSH 作为底座

DSH 是 DeepSeek 于 2026 年 8 月开源的 agent 运行时框架（MIT 协议，TypeScript/Node.js），基于 Cordis 插件框架，所有组件都是插件。救援业务以插件、MCP 服务器和技能的形式开发，不改 DSH 源码。

| DSH 特性 | 对本项目的价值 |
| --- | --- |
| 一切皆插件：模型适配器、工具注册表、agent 循环都可替换 | 数据接入、专业模型、知识库各自独立开发、测试、升级 |
| agent 循环是公开的事件协议（如 `agent/pre-step`、`step/end`） | 在事件上挂钩子，实现人工确认节点和合规拦截 |
| 只追加的执行日志 | 每条研判和建议都有完整依据链，用于审计和事故复盘 |
| Agent 预设：打包身份、规则、工具和工作流 | 研判、调度、方案、审核各自定义为一个预设 |
| `ctx.subagents` 多后端子智能体 | 总指挥 Agent 编排多个专业 Agent |

| DSH 特性 | 对本项目的价值 |
| --- | --- |
| headless 无界面模式 | 由监控报警自动触发初步研判 |
| 模型无关 | 开发用 DeepSeek API，部署时切换为本地部署的任意模型 |

**注意**：DSH 仍处于开发者预览阶段，官方提示会有不兼容变更。项目必须锁定版本，以 bundle 方式扩展 DSH、不 fork，对 DSH 接口的引用集中在一个适配文件里（见风险与对策）。

参考资料：DSH 仓库 · DSH 架构评测 · DSH 深度解析

## 总体架构

系统分四层：数据层采集矿井实时数据和台账，能力层用确定性模型计算，智能体层在 DSH 上编排各专业 Agent，交互层把建议交给指挥人员确认。




总体架构 · 四层

所有数值结论在能力层产生；智能体层只做理解、编排和文字生成，并通过事件钩子实现人工确认和合规拦截。

# 三大功能模块（全灾种总体设计）

三个模块对所有灾种通用，差异只在数据源和专业模型。一期只实现火灾部分，其余灾种按此框架扩展。

## 灾情研判

| 灾种 | 主要输入 | 研判内容 | 依托模型 |
| --- | --- | --- | --- |
| 火灾 | CO、$CO_2$、$O_2$、$CH_4$、$C_2H_4$、温度、烟雾、风速、皮带综保 | 火源位置、火势阶段、烟流影响范围、瓦斯爆炸危险性 | 标志气体分析、格雷哈姆系数、爆炸三角形、通风网络解算 |
| 水灾 | 水位、涌水量、水文观测孔、采掘工程图 | 突水水源、淹没范围与时间、被困人员可能位置 | 水化学判别、巷道容积-涌水量淹没推演 |
| 坍塌/冒顶 | 矿压、支架阻力、微震、人员定位 | 冒落范围、二次垮塌风险、被困区域供氧通风状态 | 矿压预警、经验与数值模型 |
| 通用 | 人员定位、视频、避险硐室状态 | 受困人数与位置、灾区边界 | 人员定位、视频识别 |

输出为结构化研判报告：事故等级建议、影响范围图层、危险源清单、置信度、数据依据，随监测数据滚动更新。

## 物资调度

1. 维护应急物资、装备、救护队、医疗和外部救援力量台账（位置、数量、状态、响应时间）。
2. 根据研判结果生成需求清单。
3. 用运筹优化（车辆路径问题、整数规划）计算从哪调、调多少、走哪条路、多久到，考虑井下运输条件和灾区封锁。
4. 生成调度单，经确认后推送，并跟踪到位情况。

## 处置方案

1. 以本矿应急预案和《煤矿安全规程》《矿山救援规程》《煤矿防治水细则》为依据，用检索增强（RAG）调取相关条款。
2. 结合研判结果生成方案：撤离与避灾路线、通风调整、灭火/封闭/排水措施、救护队行动、警戒和外部联动。
3. 每条措施标注依据条款和数据来源。
4. 合规审核 Agent 逐条校验，违反硬性规定的措施直接拦截并说明原因；规则引擎再用代码兜底一次。

5. 支持方案对比、人工修改，修改过程记入日志。

**核心原则**：大模型负责理解、编排和生成文字；所有数值结论（气体是否具有爆炸性、烟流多久到达、调度路线）必须由确定性模型计算，不能由大模型估算。

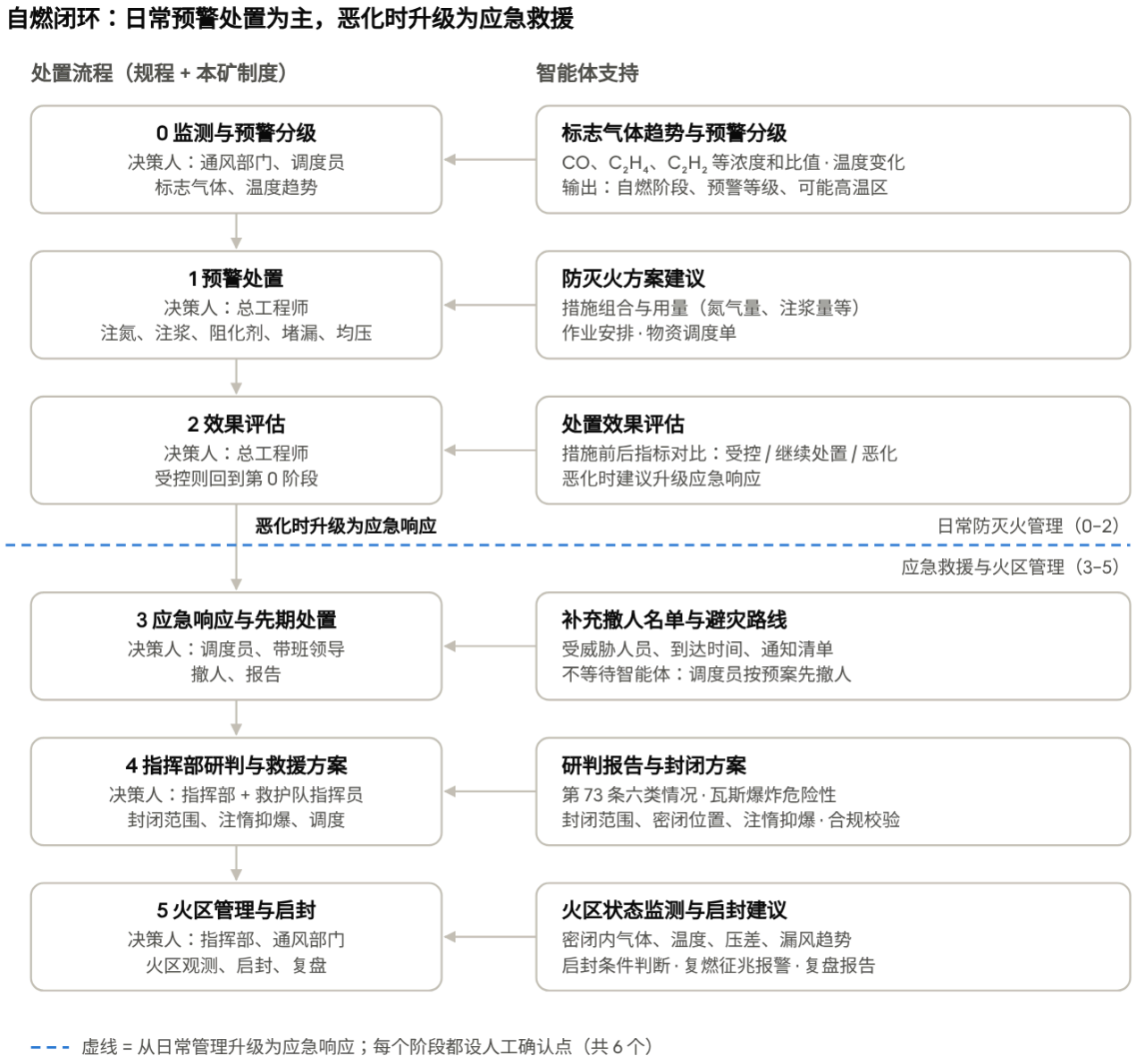
## 首个闭环场景：采空区煤自燃

一期选择采空区煤自燃作为闭环场景。它是目标客户最常见的火灾类型，而且从早期预警、防灭火处置到封闭火区、启封的全过程都能覆盖。

- **最常见**：目标客户的火灾以采空区遗煤自燃为主，演示这个场景最能引起客户共鸣；公开文献和实验数据也较丰富，便于构建模拟矿井。
- **预警期长，智能体价值大**：自燃从缓慢氧化发展到明火有一个过程，标志气体逐步出现。早发现、早处置，多数情况下能避免演变成事故。
- **数据条件较好**：开采容易自燃煤层的矿井一般建有束管监测和上隅角、回风隅角气体监测，部分矿有分布式光纤测温。
- **可扩展**：标志气体分析、爆炸危险性判定、火区管理模型和应急响应部分（撤人、指挥部研判）可直接复用到下一个场景：带式输送机巷道火灾。

**场景边界**：模拟矿井（及之后接入的客户矿井）内一个正在回采的综采工作面及其采空区，覆盖从早期预警到封闭火区、启封的全过程；不含外因火灾和其他区域。

闭环流程依据《矿山救援规程》和本矿防灭火要求设计：左侧是处置阶段与决策人，右侧是智能体的支持。与外因火灾不同，自燃闭环的重心在前段的预警和防灭火处置，多数情况下在第 2 阶段就能收住，只有恶化时才进入应急响应。




采空区煤自燃闭环 · 6 个阶段、6 个人工确认点

智能体不改变现有指挥体系，只为各阶段的决策人提供建议；一旦进入应急响应，撤人和报告由调度员按预案立即执行，不等待智能体。各阶段的具体做法见下一节。

## 采空区自燃闭环详细设计（按真实处置流程）

闭环分 6 个阶段：前 3 个阶段是日常防灭火管理中的预警和处置，后 3 个阶段是自燃恶化为火灾后的应急救援和火区管理。每个阶段先明确决策人，智能体只提供支持；规程中保障人员安全的硬性条款由合规审核 Agent 校验、规则引擎强制执行。条号指《矿山救援规程》；预警指标和防灭火措施在模拟矿井阶段参照行业通行做法和文献设定，接入真实矿井后以该矿的自然发火预测预报制度和防灭火设计为准。

## 阶段划分与智能体支持

| 阶段 | 依据 | 决策人 | 智能体做什么 | 输出 |
| --- | --- | --- | --- | --- |
| 0 监测与预警分级 | 本矿自然发火预测预报制度 | 通风部门、调度员 | 跟踪标志气体浓度和比值、温度的变化趋势，判断自燃发展阶段和预警等级，推断可能的高温区域 | 预警等级建议及依据 |
| 1 预警处置 | 本矿防灭火设计 | 总工程师（或通风副总） | 推荐防灭火措施组合（注氮、注浆、喷洒阻化剂、堵漏、均压）及用量，生成作业和物资安排 | 防灭火方案建议、物资调度单 |
| 2 效果评估 | 本矿防灭火设计 | 总工程师 | 对比措施前后的指标，判断已受控、需继续处置还是在恶化；受控则回到第 0 阶段常态监测 | 评估报告、是否升级的建议 |
| 3 应急响应与先期处置 | 第 39、40 条 | 调度员、带班领导 | 计算受威胁人员、撤人范围、避灾路线和通知清单 | 初报、补充撤人名单 |
| 4 指挥部研判与救援方案 | 第 73、83–87、98 条 | 指挥部 + 救护队带队指挥员 | 汇总第 73 条六类情况；评估瓦斯爆炸危险性；给出封闭范围、密闭位置、注惰抑爆方案和物资调度 | 研判报告、封闭方案建议 |
| 5 火区管理与启封 | 第 87、164 条 | 指挥部、通风部门 | 持续分析密闭内气体、温度、压差和漏风；判断是否满足启封条件；启封时监测复燃征兆 | 火区状态报告、启封建议、复盘报告 |

**设计约束**：一旦进入应急响应，撤人和报告由调度员按预案立即执行，不等待智能体。智能体在这一阶段只做补充（例如提示预案之外的受威胁人员），绝不能成为撤人的前置条件。

## 数据输入

| 数据源 | 内容 | 用于阶段 |
| --- | --- | --- |
| 束管监测 | 采空区和回风隅角的 $O_2$、CO、$CO_2$、$CH_4$、$C_2H_4$、$C_2H_2$ 等组分 | 0–2、5 |
| 安全监控系统 | 上隅角、回风巷的 CO、$CH_4$、温度测点 | 0–4 |
| 测温系统 | 采空区温度测点或分布式光纤测温 | 0–2 |
| 生产数据 | 工作面推进度、停采和过构造情况 | 0、1 |

| 数据源 | 内容 | 用于阶段 |
| --- | --- | --- |
| 防灭火系统 | 注氮、注浆、阻化剂系统的状态和能力 | 1、2 |
| 通风系统与矿图 | 风量风压、漏风通道、密闭位置 | 1–5 |
| 煤层参数 | 自燃倾向性、最短自然发火期、本矿标志气体及临界值 | 0 |
| 人员定位系统 | 井下人员实时位置 | 3、4 |
| 救援力量与物资台账 | 救护队、医疗、惰性气体、密闭材料 | 1、3、4 |
| 预案与规程 | 本矿火灾专项预案、防灭火设计、灾害预防和处理计划、《矿山救援规程》 | 1–5 |

## 人工确认节点

| 节点 | 阶段 | 确认人 |
| --- | --- | --- |
| 预警等级 | 0 | 通风部门负责人 |
| 防灭火方案 | 1 | 总工程师 |
| 升级为应急响应 | 2 | 总工程师或矿长 |
| 补充撤人名单与通知 | 3 | 调度员 |
| 封闭或灭火方案 | 4 | 总指挥 + 救护队带队指挥员 |
| 启封 | 5 | 总指挥 |

确认前系统只展示建议，不推送；确认操作须登录并按角色授权，只有表中对应角色能确认；每次确认、驳回和修改连同确认人身份写入执行日志。救护队行动方案由救护队制定（第 46 条），智能体只提供灾情和数据支持。

## 红线规则库（合规审核 Agent）

| 规则 | 条号 | 智能体动作 |
| --- | --- | --- |
| 采空区或冒落带火灾，保持通风系统稳定，防止瓦斯涌入火区 | 第 98 条 | 拦截可能破坏通风稳定的建议 |
| 封闭有爆炸危险的火区，先注入惰性气体等抑爆，再在安全位置建进、回风密闭 | 第 85 条 | 方案缺少抑爆步骤则拦截 |

| 规则 | 条号 | 智能体动作 |
| --- | --- | --- |
| 封闭时先建临时密闭，确认无爆炸危险后再建永久或防爆密闭 | 第 84 条 | 校验步骤顺序 |
| 封闭过程中可燃气体浓度迅速增加，所有人员立即撤离 | 第 85 条 | 实时报警 |
| 甲烷超过 2% 且上升、风量突变或出现风流逆转征兆，立即撤出 | 第 75 条 | 实时报警 |
| 火区封闭后进入检查一般在 24 小时后；密闭被爆炸破坏的，严禁派救护队探察或恢复 | 第 87 条 | 提醒并拦截 |
| 启封前检查温度、气体和支护并撤出回风侧人员；启封后 3 日内每班检查，发现复燃征兆立即重新封闭 | 第 164 条 | 校验启封方案，实时报警 |
| 反风或风流短路前，必须先撤出受影响区域人员 | 第 77 条 | 方案缺少撤人步骤则拦截 |
| 进入灾区每队不少于 6 人并设待机小队；温度超过 40°C 不得佩用氧气呼吸器作业 | 第 57、53、88 条 | 校验涉及进入灾区的建议 |

规则库还需并入本矿防灭火设计和预案中的要求，并由通风部门和救护队审定后才能上线。

### 接入真实矿井前的现场调研

- [ ] 收集客户矿井的自然发火预测预报制度（标志气体及临界值）、防灭火设计、火灾专项预案、灾害预防和处理计划，以及《煤矿防灭火细则》的相关要求
- [ ] 整理近几年的自燃预警和处置记录，作为评测数据
- [ ] 访谈通风部门、调度员和救护队带队指挥员，核对上述阶段和决策人，标出与本矿实际的差异

依据：《矿山救援规程》全文 · 规程解读

## 模拟矿井设计

模拟矿井是一期的开发、演示和评测环境。它通过与真实系统相同的数据接口为智能体供数；接入真实矿井时按模块替换数据源，智能体本身不需要修改。




模拟矿井 · 按模块替换为真实系统

## 设计原则

1. **数据契约先行**：先定义统一的测点模型、数据格式和事件格式，模拟和真实数据源都按契约输出。模拟器尽量按真实监控平台的数据输出方式供数，让对接变成"换一个数据源"。
2. **数据要像真的**：矿井模型基于典型矿井布局（设计院或高校的脱敏图纸），巷道网络参数真实；自燃过程依据文献和程序升温实验数据；注入传感器噪声、漂移、断线、缺数和误报。
3. **能响应处置**：模拟器接收注氮、注浆、堵漏、均压、封闭等处置输入，指标随之变化。否则第 2 阶段的效果评估无法演示，闭环就不成立。
4. **场景脚本化**：每个场景可重复、可加速播放，几十分钟演完几天的过程。

## 模块组成

| 模块 | 模拟实现 | 接入真实矿井时替换为 |
| --- | --- | --- |
| 矿井与通风模型 | 典型矿井巷道网络、风量风压、漏风通道 | 客户矿图和通风系统数据 |
| 自燃过程模型 | 采空区氧化升温与标志气体演化 | 无需替换，真实矿井直接用监测数据 |
| 束管与安全监控 | 按测点生成气体和温度数据，含脏数据 | 客户束管监测和安全监控平台 |
| 测温 | 采空区温度测点或分布式光纤 | 客户测温系统；没有则不启用 |
| 人员定位 | 按班次和作业计划生成人员分布 | 客户人员定位系统 |

| 模块 | 模拟实现 | 接入真实矿井时替换为 |
| --- | --- | --- |
| 防灭火系统 | 注氮、注浆等系统能力和执行反馈 | 客户系统状态数据；多为人工记录，可能需新开发填报模块 |
| 生产数据 | 推进度、停采计划 | 客户生产管理系统或人工填报 |
| 物资与救援力量台账 | 示例台账 | 客户台账；多为表格，需开发导入工具 |
| 预案与规程 | 规程原文和编写的示例预案 | 客户真实预案和防灭火设计 |

## 标准场景

1. 正常回采：基线，检验误报率。
2. 缓慢氧化，及时处置后受控。
3. 加速自燃，处置有效。
4. 处置无效，恶化为火灾，进入应急响应和封闭。
5. 封闭后火区演化与启封，含复燃。
6. 传感器故障与误报。

这 6 个场景同时用作演示脚本、评测集和回归测试集。

## 可信度保障

- 请通风或防灭火方向的专家审核场景参数和智能体输出；考虑与有实验数据和矿井资源的高校或研究院合作，他们也可能帮助引荐第一个客户。
- 对外介绍时注明指标来自模拟环境，真实效果在接入真实矿井后验证。

## 演示方案

阶段 4 的交付物之一是一条约 20 分钟的标准演示线，基于标准场景 2、4、5 加速播放。每一步都同时展示依据（数据、计算过程、规程条号）和确认人。

1. **开场（2 分钟）**：模拟矿井总览，工作面、测点和人员分布叠加在矿图上。
2. **早期预警（4 分钟）**：标志气体缓慢上升，智能体给出预警等级、依据和可能的高温区，通风负责人确认等级。
3. **防灭火处置（4 分钟）**：智能体推荐注氮、注浆组合和用量并生成调度单；总工程师确认后模拟器响应处置，指标回落，展示效果评估。
4. **恶化与升级（5 分钟）**：切换到处置无效的场景，指标加速上升，智能体建议升级为应急响应；展示补充撤人名单和避灾路线，并说明撤人不等待智能体。
5. **封闭方案与合规拦截（3 分钟）**：智能体生成封闭方案；现场演示一条故意违规的修改（例如去掉注惰抑爆步骤）被规则引擎拦截，并给出对应条号。

6. **复盘（2 分钟）**：一键生成复盘报告，展示每条建议的依据链和确认记录。

另外准备一个互动环节：让客户现场调整场景参数（如推进速度、漏风量）重新运行，证明不是预先录好的结果。演示环境要能完全离线运行，模型使用本地部署或演示前缓存的版本，避免现场网络问题。

### 接入真实矿井的三个步骤

1. **离线数据验证**：客户提供一段历史监测数据（可以脱敏），在模拟环境中回放，检验研判效果。只需提供数据，不涉及系统对接。
2. **只读旁路接入**：只从地面数据平台读取数据，不向井下系统写入任何东西，智能体只出建议、不推送。在这一步把模拟数据源逐个替换为真实数据源。
3. **试运行**：与人工指挥并行运行，按该矿制度完成预警指标、红线规则和确认流程的本地化。

## 在 DSH 上的落地方式

采用混合方式：数据接入、计算模型和知识检索做成 MCP 服务器，领域方法做成技能，安全控制和编排保留为 DSH 原生插件。划分标准是：必须插手 agent 循环的放在运行时内部，其余尽量放到运行时外部。DSH 侧的全部内容打包成一个 bundle（`dsh-bundle/`），用 `dsh plugin add` 装入 `rescue` 配置档，不 fork、不改 DSH 源码。MCP 服务器通过官方的 `@deepseek-ai/dsh-mcp-client` 接入，技能通过 `@deepseek-ai/dsh-skill-filesystem` 从仓库 `skills/` 下按角色划分的目录加载。

### 三种形式的分工

| 形式 | 放什么 | 为什么 |
| --- | --- | --- |
| DSH 原生插件 | 人工确认与合规拦截、执行日志、DSH 适配层、Agent 预设 | 只有挂在 agent 循环事件上的钩子能在工具调用执行前拦截；安全控制必须用代码强制执行，不能依赖模型自觉遵守 |
| MCP 服务器 | 数据接入（含模拟矿井）、计算模型、知识检索 | 模拟矿井换真实矿井只换服务器配置；科学计算可以用 Python；独立进程，故障隔离；还能被其他系统复用 |
| 技能（SKILL.md） | 研判方法、方案编写规范、报告模板、规程要点 | 按需加载，节省上下文；领域专家可以直接审阅和修改，不用改代码 |

MCP 工具接入后挂在同一个工具注册表里，所以人工确认钩子同样能拦截 MCP 工具的调用。

## 模块清单（一期）

| 模块 | 形式 | 职责 |
| --- | --- | --- |
| `rescue-approval` | 原生插件 | 指令类工具执行前先调 actions 的红线预检（预检不可达也拒绝），通过后发起人工确认 |
| `rescue-audit` | 原生插件 | 记录每次指令类工具调用的结果（已发布、被拦截、失败），供复盘 |
| `contract` | Python 库 | 数据契约、红线引擎、方案校验、mine-data 客户端库和一致性测试，各服务共用 |
| `mine-data-sim` | MCP 服务器（Python） | 模拟矿井：矿井与通风模型、自燃过程模型、传感器仿真、场景脚本；按数据契约提供监测、人员定位、防灭火系统和台账数据 |
| `mine-data-<矿名>` | MCP 服务器 | 真实矿井适配器，接入时开发，实现与 `mine-data-sim` 相同的接口；可按数据源拆成多个 |
| `ventnet` | MCP 服务器（Python） | 巷道拓扑、通风网络解算、漏风分析、避灾路径 |
| `fire-model` | MCP 服务器（Python） | 标志气体趋势、数据质量检测、预警分级、爆炸三角形、复燃和甲烷实时报警 |
| `dispatch` | MCP 服务器（Python） | 防灭火用量计算、物资调度优化、调度单草案 |
| `kb` | MCP 服务器（Python） | 知识检索封装层：调用 RAGFlow，统一检索参数和引用格式；RAGFlow 不可用时退回本地红线条文；提供方案红线自查 `check_plan_redlines` |
| `actions` | MCP 服务器（Python） | 唯一的指令出口：发布预警、下发方案和调度单、撤人通知、升级、启封审批；服务端再校验一次并写指令台账；模拟环境下把确认后的处置转发给模拟矿井 |

计算类服务器通过共用的数据契约客户端库直接取数，不经过模型传递大量原始数据；模型只传测点、时间范围等参数，拿回计算结论和过程。原计划的 `rescue-adapter` 插件改为 `dsh-bundle/src/adapter/dsh.ts` 一个文件：插件对 DSH 类型的引用都集中在这里，升级时只改这一处。

## 知识库：RAGFlow + 封装层

知识库使用其他部门本地部署的 RAGFlow。一期就在它前面加一层薄封装 `kb`（Python MCP 服务器）：智能体只调用 `kb`，不直接连接 RAGFlow。这样大部分调整可以在封装层自己完成，减少跨部门沟通。

| 事项 | 由谁负责 |
| --- | --- |
| 工具形式：按用途提供 `search_regulations`（法规规程）、`search_mine_plans`（矿井预案）、`search_cases`（文献案例）、`get_article`（按文档和条号取原文） | 封装层 |
| 知识库映射：工具与 RAGFlow 知识库的对应关系；模拟矿井和客户矿井之间切换预案库 | 封装层 |
| 检索参数：返回条数、相似度阈值、关键词权重、是否重排 | 封装层（在 RAGFlow 已开放的范围内） |
| 查询改写：补充行业术语和同义词（如"上隅角""束管"），按灾种加限定词 | 封装层 |
| 结果处理：统一引用格式（文档名、条号、版本），去重，过滤已废止文件 | 封装层 |
| 缓存与降级：缓存高频检索；红线规则相关条款在本地留存，RAGFlow 不可用时合规校验照常进行 | 封装层 |
| 日志：记录每次检索的问题、命中结果和耗时，写入执行日志 | 封装层 |
| 文档上传、解析和切片方式、嵌入模型、知识库创建与授权 | RAGFlow 管理方；如对方同意开放文档管理接口，上传和重新解析也可由封装层完成 |

- **连接方式**：封装层通过 RAGFlow 的 HTTP 检索接口调用，而不是再套一层它自带的 MCP 服务器，这样能用上完整的检索参数。RAGFlow 升级导致接口变化时，也只需修改封装层。
- **账号**：为智能体单独申请一个只读、只授权救援知识库的 RAGFlow 账号，API Key 只保存在封装层。
- **知识库划分**：法规规程库（《矿山救援规程》《煤矿安全规程》《煤矿防灭火细则》等）、矿井预案库（模拟阶段放示例预案，接入后换成客户真实预案和防灭火设计）、文献案例库（事故调查报告、研究文献）。规程类文档按条切分，便于引用条号，切片方式需与 RAGFlow 管理方约定。

```
# .env 片段：kb 封装层读取的 RAGFlow 配置
RAGFLOW_BASE_URL=http://ragflow.internal:9380
RAGFLOW_API_KEY=<只读账号的 Key>
KB_DATASETS_REGULATIONS=<法规规程库 id>
KB_DATASETS_PLANS=<矿井预案库 id>      # 接入客户矿井时改为该矿的预案库
KB_DATASETS_CASES=<文献案例库 id>
```

### 技能清单（一期）

| 技能 | 内容 |
| --- | --- |
| `fire-warning-assessment` | 预警分级方法、标志气体解读、何时调用哪个计算工具 |
| `emergency-briefing` | 按第 73 条六类情况汇总灾情的方法和模板 |
| `firefighting-plan` | 防灭火措施的选择原则和方案结构 |
| `sealing-and-unsealing` | 封闭火区和启封的步骤与注意事项 |
| `report-templates` | 初报、研判报告、复盘报告的格式 |
| `red-line-rules` | 红线规则说明，让模型提前避开违规方案；强制执行由同一套红线引擎在 `rescue-approval` 预检和 actions 服务端两处完成 |

### Agent 预设

- **总指挥协调 Agent**（`rescue-commander`，默认）：工具 `mine_data`、`fire_model`、`actions`，以及委派、向人提问、待办；接收预警和报警，按阶段委派各专业 Agent，汇总输出。它是唯一挂载 `actions`、能发出指令的 Agent。
- **火灾研判 Agent**（`rescue-assessment`）：工具 `mine_data`、`fire_model`、`ventnet`；技能 `fire-warning-assessment`、`emergency-briefing`。
- **物资调度 Agent**（`rescue-dispatch`）：工具 `mine_data`、`dispatch`。
- **处置方案 Agent**（`rescue-plan`）：工具 `mine_data`、`ventnet`、`kb`；技能 `firefighting-plan`、`sealing-and-unsealing`；方案写完先调用 `kb` 的 `check_plan_redlines` 自查。
- **合规审核 Agent**（`rescue-compliance`）：工具 `kb`；技能 `red-line-rules`。它的审核结论之外，同一套红线引擎还在预检和 actions 服务端用代码强制执行。
- 所有 Agent 共用 `report-templates` 技能（`skills/shared/`）。

## 目录结构

```
rescue-agent/
├─ docs/                      # 本方案、架构说明、设计决策（ADR）、DSH 验证清单
├─ dsh-bundle/                # DSH bundle（TypeScript），不 fork DSH
|  ├─ src/approval/           # rescue-approval：红线预检 + 人工确认
|  ├─ src/audit/              # rescue-audit：审计日志
|  ├─ src/adapter/            # 对 DSH 接口的引用集中在这里
|  ├─ presets/                # 5 个 Agent 预设
|  └─ cordis.patch.yml
├─ mcp/                       # MCP 服务器（Python，uv 工作区）
|  ├─ contract/               # 数据契约、红线引擎、客户端库、一致性测试
|  ├─ mine-data-sim/          # 模拟矿井
|  ├─ ventnet/
|  ├─ fire-model/
|  ├─ dispatch/
|  ├─ kb/                     # RAGFlow 封装层 + 红线自查
|  └─ actions/                # 指令出口 + 指令台账
├─ skills/                    # 技能，按角色分目录，每个技能一个 SKILL.md
├─ scenarios/                 # 6 个标准场景
├─ eval/                      # 基线评测
├─ profiles/                  # 本地大模型、真实矿井环境变量模板
└─ scripts/                   # 安装、启动、冒烟、模拟控制台
```

`mcp/contract/` 里的一致性测试对模拟和真实数据源跑同一套用例，新写的真实矿井适配器通过测试才能替换上线。

## 配置与代码示意

接入一个 MCP 服务器是在 Agent 预设里插入一个 `dsh-mcp-client` 实例。所有服务都用 streamable-http，地址和令牌从环境变量读取，部署时不改配置文件：

```
# dsh-bundle/presets/plan.patch.yml 片段
- id: mcp-kb
  name: '@deepseek-ai/dsh-mcp-client'
  config:
    serverName: kb
    transport: streamable-http
    url: !!js "process.env.RESCUE_MCP_KB_URL ?? 'http://127.0.0.1:8105/mcp'"
    headers: !!js "process.env.RESCUE_MCP_TOKEN ? { Authorization: 'Bearer '
+ process.env.RESCUE_MCP_TOKEN } : {}"
    toolCallTimeoutMs: 60000
```

接入真实矿井时，只把 `RESCUE_MINE_DATA_MODULE`（同机部署）或 `RESCUE_MCP_MINE_DATA_URL`（指向矿区内网服务）换成该矿的适配器，`serverName` 保持不变，智能体和技能都不用改。

计算服务用 Python 的 MCP SDK 2.x 编写（2.x 的服务类是 `MCPServer`，已没有 FastMCP 这个名字）：

```
# mcp/fire-model/src/rescue_fire_model/server.py（节选）
from mcp.server.mcpserver import MCPServer
from rescue_contract.client import MineData      # 数据契约客户端库
from . import explosion

def build_server() -> MCPServer:
    md = MineData.from_env()
    server = MCPServer("fire_model")

    @server.tool()
    async def assess_explosion_risk(ch4_pct: float, o2_pct: float) ->
dict[str, Any]:
        """用爆炸三角形判定甲烷–空气混合气体的爆炸危险性"""
        risk = explosion.classify(ch4_pct, o2_pct)   # 确定性计算，返回结论和计算
过程
        return {"explosion_risk": risk.value, ...}

    return server

# __main__.py：serve(build_server(), "fire_model")  → streamable-http，默认端口
8103
```

指令统一走 `actions` 服务，设三道防线：

1. **红线预检**：`rescue-approval` 在工具调用执行前的 `tools/pre-execute` 事件上挂钩子，把方案发给 actions 的 `/redlines/check`；不通过直接拒绝并给出条号，预检服务不可达也拒绝。
2. **人工确认**：预检通过后弹出确认，指挥员确认后才放行。
3. **服务端终检**：actions 收到调用后，用同一套红线引擎再查一次方案类型、数据时效和红线，不信任调用方；通过才写指令台账。

红线规则只在 `rescue_contract.redlines` 实现一次，方案 Agent 的自查、插件预检和服务端终检共用；规则说明技能与规则表有同步测试。

## 混合方式的代价

- **调用延迟**：多了进程间调用，每次工具调用略慢；对分钟级的研判没有影响。
- **认证**：DSH 的 MCP 客户端只发送静态请求头，不支持 OAuth；矿区内网用静态令牌即可。
- **运维**：多了几个服务进程，用容器统一编排和监控。

## 工程骨架现状

工程骨架已搭好，不依赖大模型的部分全部可运行：Python 测试 71 个、插件测试 15 个通过，6 个服务冒烟通过，模拟矿井上“红线自查 → 下发方案 → 写台账 → 模拟器响应处置”已跑通，6 个标准场景的确定性基线评测全部达到当前期望。代码仓库的 `README.md` 说明了目录分工和可并行的工作线。

搭建中对初版方案做了以下调整（本文各节已同步，理由见仓库 `docs/decisions/`）：

| 调整 | 初版 | 现在 | 原因 |
| --- | --- | --- | --- |
| DSH 扩展方式 | fork DSH，插件放 `plugins/` | 一个 bundle（`dsh-bundle/`）装入 `rescue` 配置档 | 升级不用合并上游代码 |
| 适配层 | 独立插件 `rescue-adapter` | 一个文件 `src/adapter/dsh.ts` | 只需集中类型引用，不需要运行时组件 |
| 指令出口 | 分散在各模块，确认只在 DSH 侧 | 新增 `actions` 服务，三道防线 | 绕过 DSH 也有保护，指令有统一台账 |
| 红线引擎 | 在 `rescue-approval` 里 | Python 契约包 `rescue_contract`，三处共用 | 规则只写一份，不会漂移 |
| 方案自查工具 | — | 放在 `kb` | 方案和合规 Agent 能自查，但不获得下指令的能力 |
| MCP 连接 | stdio，FastMCP | streamable-http，地址走环境变量；SDK 2.x 的 `MCPServer` | 服务可分机部署、独立测试；2.x 已无 FastMCP |
| 技能目录 | `.dsh/skills/` 自动发现 | `skills/<角色>/`，每个预设只加载本角色和公共技能 | 各 Agent 只看到自己用得上的技能 |

尚未完成或需要确认的事项：

- [ ] 在联网环境装上 DSH 0.2.1-rc.4，按仓库 `docs/validation-checklist.md` 的 17 项验证（插件只对照接口签名做过类型检查）。
- [ ] 专家审定预警阈值、场景参数、技能内容、评测期望（目前都是开发占位值）。
- [ ] 复核传感器漂移的交叉校核容差：基线评测发现场景 6 误报后已收紧，需用现场数据确认不会把真实征兆排除。
- [ ] 确认人按角色限定（依赖 DSH 账号和角色能力）。
- [ ] 通风网络解算（Hardy Cross）、智能体端到端评测。

# 大模型接入

智能体与具体模型解耦：开发阶段使用 DeepSeek API；部署阶段改为其他部门在本地部署的模型，在 DSH 中配置为自定义模型提供方即可，不限于 DeepSeek 模型。

## 分阶段使用

| 阶段 | 模型来源 | 数据要求 |
| --- | --- | --- |
| 开发与模拟演示 | DeepSeek API | 只处理模拟矿井数据和公开规程、文献 |
| 客户离线数据验证 | 本地部署的模型 | 客户数据原则上不出内网；本地模型尚未就绪时，须经客户书面同意并脱敏后才能使用 API |
| 只读接入与试运行 | 本地部署的模型 | 矿区内网，数据不出网 |

## 对本地模型的要求

以下要求在阶段 0 提给模型部署部门：

- **接口**：采用 DSH 自定义模型提供方支持的协议（通常是 OpenAI 兼容接口）。
- **工具调用**：必须稳定支持工具调用（function calling）。智能体的所有数值结论都靠调用工具获得，这是硬性要求。
- **上下文长度**：建议不低于 64K，以容纳研判数据摘要、规程检索结果和多轮对话。
- **并发**：能支撑 5 个 Agent 同时工作，以及演示和多人同时使用；具体容量按评测结果与部署部门确定。
- **稳定性**：约定服务可用性要求，模型版本变更提前通知。

## 模型准入测试

任何模型上线前，包括更换模型和升级版本，都要跑同一套准入测试：工具调用正确率、结构化输出格式合规率、6 个标准场景的端到端结果、红线违规建议的比例。各项指标不低于开发阶段 DeepSeek API 的基线，才能切换。

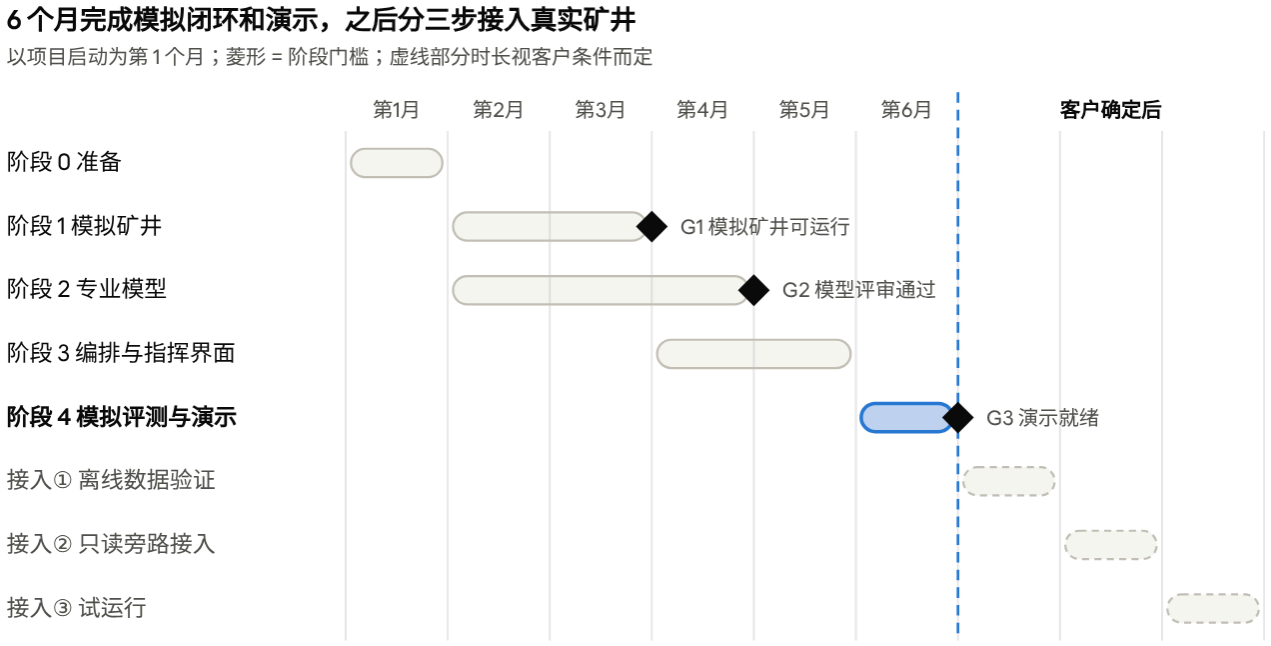
本地模型能力可能弱于开发时用的 API。由于人工确认和红线拦截都在代码里，模型变弱只会影响建议质量，不会突破安全底线；必要时可以通过调整技能内容或 Agent 分工来弥补。

## 模型不可用时的降级

模型服务中断时，确定性部分照常运行：监测数据、计算模型和红线报警不依赖大模型，告警直接推送给值班人员。文字研判和方案生成暂停，并在界面上明确提示，人员按纸质预案处置。

# 实施步骤与里程碑

一期约 6 个月完成模拟矿井闭环和客户演示，设 3 个门槛；客户确定后分三步接入真实矿井，时长视客户条件而定。之后再按带式输送机巷道火灾、水灾、坍塌的顺序扩展场景。




实施路线图 · 模拟闭环与演示，之后分三步接入真实矿井

各阶段交付物：

1. **阶段 0 准备**：组建团队（AI/软件 + 通风、防灭火专家顾问）；锁定 DSH 版本（以 bundle 扩展，不 fork），跑通本地模型；定义数据契约；与 RAGFlow 管理方约定知识库划分、切片方式和只读账号；向模型部署部门提出本地模型要求；收集规程、文献、实验数据和典型矿井图纸。
2. **阶段 1 模拟矿井**：`mine-data-sim` 实现矿井与通风模型、自燃过程模型、传感器仿真（含脏数据）、处置响应接口和 6 个标准场景；预案和规程导入 RAGFlow；建立示例台账。
3. **阶段 2 专业模型**：标志气体趋势与预警分级、高温区推断、防灭火用量计算、爆炸三角形、通风网络解算与漏风分析、避灾路径，用模拟场景和文献数据验证，并请专家评审。
4. **阶段 3 编排与指挥界面**：5 个 Agent 预设、6 个技能、人工确认和合规拦截钩子、结构化输出模板、叠加矿图的指挥界面。
5. **阶段 4 模拟评测与演示**：跑评测集，专家评审，打磨演示版本和演示脚本（见"演示方案"）。
6. **真实矿井接入**：离线数据验证、只读旁路接入、试运行三步（见"模拟矿井设计"）。

**扩展顺序**：带式输送机巷道火灾复用气体分析、通风网络和应急响应部分，只需补充综保数据和烟流蔓延模型，增量最小，放在第一位；水灾需新增水文数据和淹没推演模型；坍塌依赖矿压和微震数据，放在最后。每个新场景只新增数据和计算类 MCP 服务器及相应技能，编排、确认、审核、日志直接复用。

### 团队配置（草拟）

一期（6 个月到演示）核心团队约 8–12 人，另配 2 名兼职的外部领域专家。

| 角色 | 人数 | 主要工作 | 主要投入阶段 |
| --- | --- | --- | --- |
| 项目负责人 / 产品 | 1 | 需求、进度、客户沟通、演示 | 全程 |
| 智能体与平台开发（TypeScript） | 1–2 | DSH 定制、原生插件、Agent 预设、技能的工程化 | 0、3 |
| 算法与计算模型（Python） | 2–3 | 通风解算、自燃模型、预警分级、调度优化、`kb` 封装层 | 1、2 |
| 模拟矿井与数据工程 | 1–2 | 数据契约、`mine-data-sim`、传感器仿真、场景脚本 | 0–2 |
| 前端（指挥界面） | 1–2 | 矿图态势、确认界面、演示界面 | 3、4 |
| 测试与评测 | 1 | 评测集、回归测试、模型准入测试、演示验证 | 2–4 |
| 运维（可兼职） | 0.5–1 | 容器编排、环境部署，对接模型和 RAGFlow 部署部门 | 全程 |
| 领域专家顾问（外部兼职） | 2 | 通风/防灭火、矿山救护各 1 人：审核场景、技能和红线规则 | 全程按需 |

- **人员紧张时**：可以合并角色（例如模拟矿井与算法合并、测试由开发轮值），最小可行团队约 6 人，周期相应延长到 8–9 个月。
- **接入真实矿井时**：视客户系统数量增加 1–2 名对接开发，负责编写真实矿井适配器和填报、导入工具。
- **领域专家不能省**：场景参数、技能内容和红线规则都需要专家把关，这直接决定演示的可信度。

## 测试评估与验收标准

分两步验收：模拟阶段以评测集成绩和专家评审为准，用于客户演示；接入真实矿井后以离线数据验证和试运行结果为准。指标目标值为建议值，需与客户和救护队共同确定。

### 评测集

- **模拟场景**：模拟矿井的 6 个标准场景及其参数变体。
- **专家想定**：由通风、救护专家编写的场景，覆盖不同自燃发展速度、高温区位置、瓦斯条件和人员分布，并包含误报场景（传感器故障、粉尘干扰）。

- **文献案例重建**：根据公开的事故调查报告和文献案例重建的过程数据。
- **客户历史数据**：接入阶段由客户提供的历史监测数据回放。
- **演练实测**：试运行阶段结合客户矿井的应急演练，系统与人工指挥并行运行。

### 指标

| 指标 | 说明 | 建议目标 |
| --- | --- | --- |
| 预警等级判定准确率 | 预警等级与专家判定一致的比例 | 待与客户确定 |
| 提前预警时间 | 预警时间早于现行人工判定的时长 | 待与客户确定 |
| 初报出具时间 | 升级为应急响应到初报生成 | 3 分钟以内 |
| 受威胁人员召回率 | 应撤离人员被识别出的比例 | 100%（不允许漏报） |
| 方案合规率 | 方案措施通过规程校验的比例 | 100% |
| 关键措施遗漏率 | 专家清单中关键措施未出现的比例 | 0 |
| 专家综合评分 | 研判、调度、方案的可用性打分 | 待定评分表 |

受威胁人员召回率、方案合规率、关键措施遗漏率为一票否决项。

## 风险与对策

最大的风险是错误建议被采纳，因此对策的核心是人工确认、确定性计算和独立审核三道防线。

| 风险 | 对策 |
| --- | --- |
| 错误建议被直接执行 | 系统只做辅助决策；所有指令经指挥员确认；始终保留回退到纸质预案的通道 |
| 大模型幻觉 | 数值结论只来自工具计算；每条措施附出处；合规审核 Agent 独立把关；低置信度时明确提示"需人工判断" |
| 灾后数据失真 | 传感器损坏、断电时做数据质量检测，标注数据缺失区域；支持人工录入现场侦察信息 |
| DSH 版本不兼容变更 | 锁定版本，以 bundle 扩展、不 fork；对 DSH 接口的引用集中在 `src/adapter/dsh.ts`；升级前跑 `docs/validation-checklist.md`，升级只改适配文件 |

| 风险 | 对策 |
| --- | --- |
| 接口对接困难 | 接入真实矿井前摸清各系统厂家协议；优先使用监控系统已有的数据汇聚平台 |
| 网络与部署 | 矿区内网私有化部署；考虑信创硬件与等保要求；指挥中心关键节点冗余 |
| 与现行制度冲突 | 尽早与集团安全部门和救护队对接，明确系统在应急指挥流程中的角色 |
| 模拟与真实差距大 | 模拟器注入脏数据；坚持数据契约和适配层；对接阶段工作量留足余量 |
| 模拟结果被误当作真实效果 | 对外注明指标来自模拟环境；真实效果在离线数据验证和试运行中确认 |
| RAGFlow 由其他部门运维 | 通过 kb 封装层隔离接口和配置变化；红线相关条款本地留存；与对方约定变更通知和服务可用性要求 |
| 本地模型能力弱于开发时的 API | 上线前跑模型准入测试；安全控制全部在代码中，不依赖模型能力；必要时调整技能内容或 Agent 分工 |